OpenSSL Practical Lab

Individual lab report covering Questions 1 to 5.4 · **Omar Ait Said** · UM6P – College of Computing

Mission 2 — Symmetric Encryption

Challenge 1 – Question 1

Hex view of the first 64 bytes of `classified.enc`. The file opens with the ASCII tag `Salted__` and an 8-byte salt (`57 5A B8 CF 60 19 AC 39`), which is the header written by `openssl enc`. The next three 16-byte rows are exactly the same, an early sign that the cipher runs in ECB mode.

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec> powershell -Command "Format-Hex -Path classified.enc | Select-Object -First 4"


           Path: C:\Users\OMAR\Desktop\CC\Info Sec\classified.enc

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000000   53 61 6C 74 65 64 5F 5F 57 5A B8 CF 60 19 AC 39  Salted__WZ¸Ï`.¬9
00000010   77 75 1C 07 8D DF 54 B3 8A 26 B7 F6 F4 A9 CE F0  wu..ßT³&·öô©Îð
00000020   77 75 1C 07 8D DF 54 B3 8A 26 B7 F6 F4 A9 CE F0  wu..ßT³&·öô©Îð
00000030   77 75 1C 07 8D DF 54 B3 8A 26 B7 F6 F4 A9 CE F0  wu..ßT³&·öô©Îð

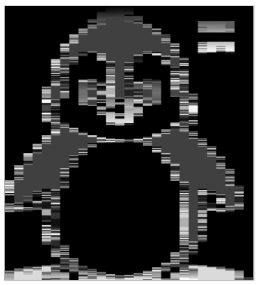

C:\Users\OMAR\Desktop\CC\Info Sec>
```

Challenge 2 – Questions 2.1 and 2.2

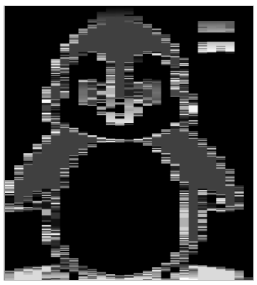
A script splits the ciphertext into 16-byte blocks and counts duplicates: 12,086 repeated blocks are found. Drawing the ciphertext bytes as a grayscale picture makes the shape of the hidden image visible, because ECB maps identical plaintext blocks to identical ciphertext blocks.

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> python ecb_pattern.py
Repeated block occurrences: 12086
Saved pattern_ecb.png

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```



Q2.1 – `pattern_ecb.png` produced by the script



Q2.2 – Ciphertext rendered as an image: the penguin outline leaks through

Challenge 3

The file is decrypted with AES-256-ECB and PBKDF2 key derivation. The `-p` option prints the salt and the derived key; the salt shown matches the one read in the file header. A second script then rebuilds the picture from the 205,200 recovered bytes.

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> "C:\Program Files\OpenSSL-Win64\bin\openssl.exe" version
OpenSSL 4.0.3 29 Sep 2026 (Library: OpenSSL 4.0.3 29 Sep 2026)

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> "C:\Program Files\OpenSSL-Win64\bin\openssl.exe" enc -d -aes-256-ecb -
pbkdf2 -p -in classified.enc -out recovered.raw
enter AES-256-ECB decryption password:

salt=575AB8CF6019AC39
key=103518D2C327E91924A746FA172211D62AC098CD2BA94E2270CC1D3612D6F877

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> python recover_image.py
Recovered bytes: 205200
Saved recovered.png

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```



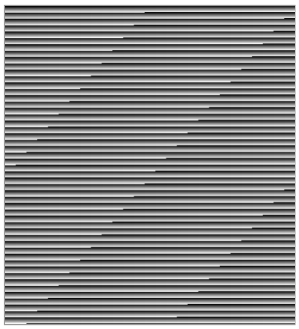
`recovered.png` – decrypted image

Challenge 4 – Questions 4.1 and 4.2

The same block analysis is run on the CBC-encrypted file: all 12,826 blocks are distinct and no repetition is detected. Once rendered, the ciphertext no longer reveals the penguin, since each block is chained to the previous one before encryption.

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> python ecb_pattern.py
Total blocks: 12826
Unique blocks: 12826
Repeated block occurrences: 0
Saved pattern_cbc.png

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```



Q4.2 – `pattern_cbc.png`: no trace of the original image

Mission 3 — RSA with Small Keys

Questions 5.1 and 5.3

Contents of the 512-bit RSA private key file `mySmall.pem`, stored in PEM format (PKCS#8).

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> type mySmall.pem
-----BEGIN PRIVATE KEY-----
MIIBVQIBADANBgkqhkiG9w0BAQEFAASCAT8wggE7AgEAAkEArE3qlygmXn1T0t1f
lUflAElMRZWKbqhH2MBjul9I2fXYwidnD06pkFpidKIqi4pBXq5pWJcrHX4LOabw
Ucbe5QIDAQABAkBlL9C/eiWmQeVzJOwYB8OVOnZfOuWz/IFM6hbnAUQQMxiMYxOP
dDzatxdYTj0QzDIPaHCKXYzM/07zf/+fJJYhAiEA2O+IjuzGzFnNArHayH95W5SA
AEMlyezM/8YqQ7Ihfz0CIQDLVO81j0dtlGSxR7oha4PCIhlJDQxH6soKsbFtZIFY
yQIgW572/t4zrBT1SAET1QA5ohsa4IF7kUfI4PGx4ft2uEUCIQCdc4bM5cd8/mU+
tnlWucfMw9TqsZTp99E+mLz5JdPXqQIhAIFOu4x1INIH8mwn72vPKxOyLLM+okrU
K692j9gAwXKu
-----END PRIVATE KEY-----

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```

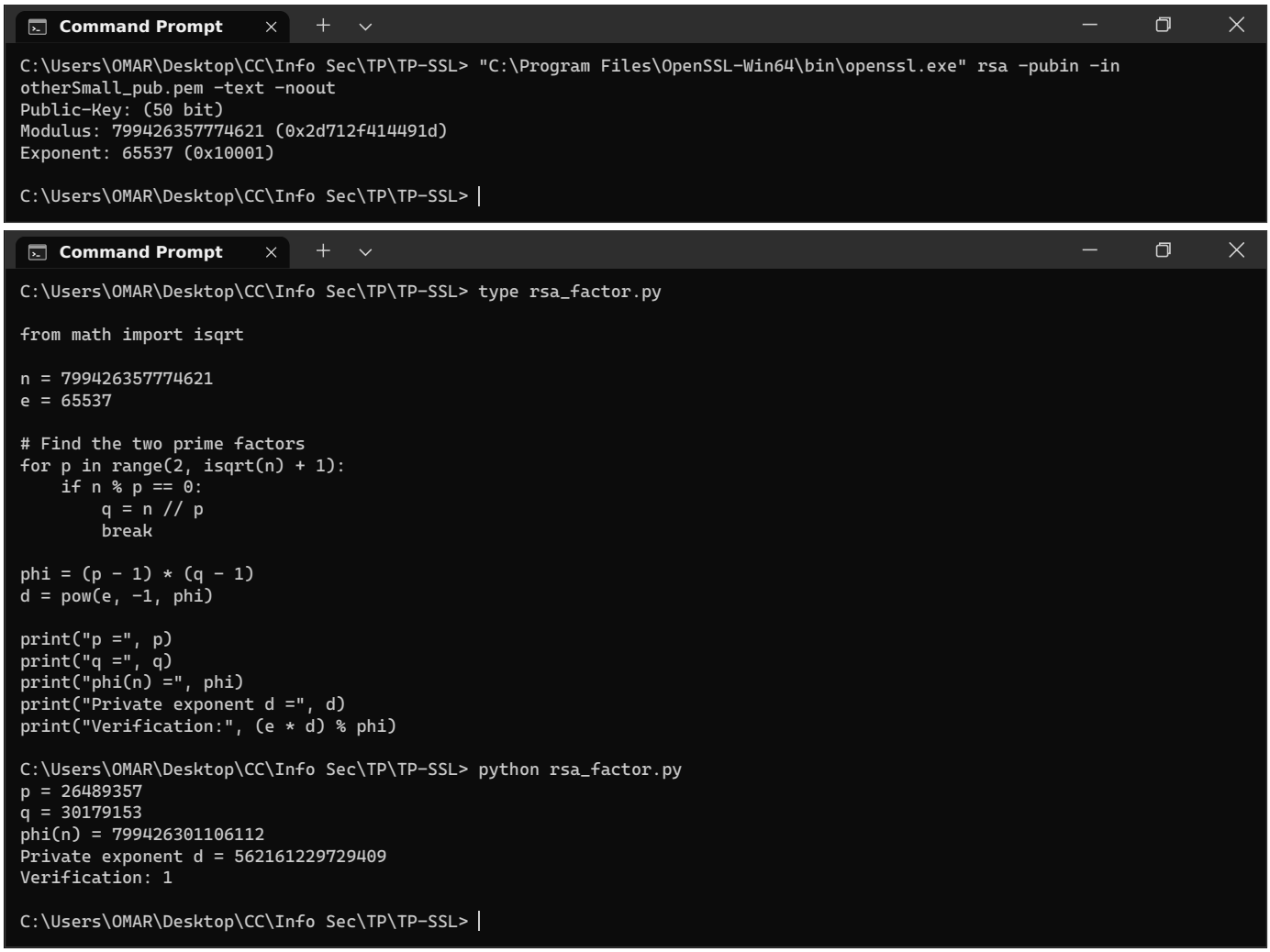
Question 5.2

Decoding the key with `openssl rsa -text` shows its fields: the 512-bit modulus, the public exponent 65537 and the private exponent.

```
Command Prompt
C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> "C:\Program Files\OpenSSL-Win64\bin\openssl.exe" rsa -in mySmall.pem -text
-noout
Private-Key: (512 bit, 2 primes)
modulus:
    ac:4d:ea:97:28:26:5e:7d:53:d2:dd:5f:95:47:e5:00:
    49:4c:45:95:8a:6e:a8:47:d8:c0:63:ba:5f:48:d9:f5:
    d8:c2:27:67:0f:4e:a9:90:5a:62:74:a2:2a:8b:8a:41:
    5e:ae:69:58:97:2b:1d:7e:0b:39:a6:f0:51:c6:de:e5
publicExponent: 65537 (0x10001)
privateExponent:
    65:2f:d0:bf:7a:25:a6:41:e5:73:24:ec:18:07:c3:95:
    3a:76:5f:3a:e5:b3:fc:81:4c:ea:16:e7:01:44:10:33:
    18:8c:63:13:8f:74:3c:da:b7:17:58:4e:3d:10:cc:32:
    0f:68:70:8a:5d:8c:cc:ff:4e:f3:7f:ff:9f:24:96:21
```

Question 5.4

The other party's public key uses a 50-bit modulus, $n = 799426357774621$. A modulus this small is broken by plain trial division: `rsa_factor.py` recovers p, q, $\varphi(n)$ and the private exponent d, then checks that $e \cdot d \bmod \varphi(n) = 1$.



```
Command Prompt

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> "C:\Program Files\OpenSSL-Win64\bin\openssl.exe" rsa -pubin -in
otherSmall_pub.pem -text -noout
Public-Key: (50 bit)
Modulus: 799426357774621 (0x2d712f414491d)
Exponent: 65537 (0x10001)

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```

```
Command Prompt

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> type rsa_factor.py

from math import isqrt

n = 799426357774621
e = 65537

# Find the two prime factors
for p in range(2, isqrt(n) + 1):
    if n % p == 0:
        q = n // p
        break

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

print("p =", p)
print("q =", q)
print("phi(n) =", phi)
print("Private exponent d =", d)
print("Verification:", (e * d) % phi)

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL> python rsa_factor.py
p = 26489357
q = 30179153
phi(n) = 799426301106112
Private exponent d = 562161229729409
Verification: 1

C:\Users\OMAR\Desktop\CC\Info Sec\TP\TP-SSL>
```